

Pair D - Traffic Lights + Intersections

Adapted from Yakov's Codebase Reading and Defence Guide (Sept 2026), split per pair by Henry. Full PDF: [docs/COMP2000_Traffic_Simulation_Codebase_Guide.pdf](#) .

You are: Addrita, Cam. **You own:** the signal state machine and the adaptive intersection logic that decides when to hold green.

Files you own

```
src/trafficsim/model/
  road/
    Intersection.java      <- sealed abstract; permits SignalisedIntersection, Roundabout
    SignalisedIntersection.java <- your main class; owns the adaptive extension logic
    Roundabout.java       <- present but unused (roundabouts removed from scope)
  light/
    TrafficLight.java      <- controls two axes (horizontal + vertical)
    LightState.java        <- state interface
    GreenState.java        <- transitions to Yellow
    YellowState.java       <- transitions to Red
    RedState.java          <- transitions to Green on the other axis
  util/
    LightPhase.java, Axis.java <- enums you consume
```

Axis-based lights

`TrafficLight` controls **axes**, not individual directions. `EAST` and `WEST` are horizontal; `NORTH` and `SOUTH` are vertical. One axis owns the active cycle while the perpendicular axis is red.

```
Horizontal GREEN -> Horizontal YELLOW -> ALL RED
-> Vertical GREEN -> Vertical YELLOW -> ALL RED
-> Horizontal GREEN
```

All-red is a safety interval between cycles - the perpendicular axis doesn't get green until every vehicle should have cleared the box.

State pattern

State class	update(...) behaviour
GreenState	When timer reaches greenDuration , set state to YellowState .
YellowState	When timer reaches yellowDuration , set state to RedState .
RedState	When timer reaches redDuration , swap axis and set GreenState .

TrafficLight increments timer first, then delegates the transition decision to the current state object. setState resets timer to zero. This is textbook State pattern - each phase knows its own transition rule.

Adaptive green extension

SignalisedIntersection watches the final four ticks of green. If at least four vehicles are stopped near the intersection on the green axis, it rewinds the timer by 12 ticks. Capped at three extensions per green phase so the perpendicular axis still gets a turn.

```
remaining = greenDuration - timer;
if (remaining <= 4 && queued >= 4 && extensions < 3) {
    timer = Math.max(0, timer - 12);
}
```

Why the caps? Without extensions < 3 , a permanently busy road could hold green forever and starve the crossing traffic. Without queued >= 4 , we'd extend for a single stopped vehicle - wasteful.

Emergency vehicle interaction

Your lights don't get pre-empted by emergency vehicles. EmergencyVehicle.stopsAtRedLight() returns !sirenOn , so the vehicle chooses to ignore the red - your light stays red on its own schedule. Normal traffic still yields via TrafficSensor.nearestEmergencyPriorityStop .

Where you touch other pairs

- **Pair A (Engine):** Their tick loop calls intersection.updateSignals() in the Lights phase. Their TrafficSensor.nearestSignalStopAhead reads light.phaseFor(direction) in the Sense phase.
- **Pair B (Network):** Your SignalisedIntersection is stored in RoadNetwork . NetworkLoader.wireIntersections connects your intersection to every road whose centre passes through its coordinates.
- **Pair C (Vehicles):** No direct call. Vehicles read light state through SensorReading.distToRedStop - they never touch your classes.

Your defence questions

Why use a State pattern for signals? Green, yellow and red have different transition rules. Separate state objects keep each rule small and avoid a giant `switch` in `TrafficLight`. Adding a new phase (e.g., flashing amber) means one new class, not editing every conditional.

Why axis-based, not per-direction? EAST and WEST should always be in the same phase - they don't conflict physically. Modelling by axis (horizontal / vertical) halves the state we track and matches how real signals work.

Why the all-red interval? Safety. When horizontal yellow ends, there may still be vehicles inside the box. All-red gives them time to clear before the vertical axis goes green. Without it, a vertical car with green could enter while a horizontal car is still crossing.

How does adaptive extension work, and why the caps? In the last 4 ticks of green, if at least 4 vehicles are queued on the green axis, we rewind the timer by 12 ticks. Capped at 3 extensions per green phase. Without the extensions cap a busy road would starve the crossing traffic; without the queue cap we'd extend for a single car.

Do emergency vehicles change your lights? No. Lights run their own schedule. The emergency vehicle chooses to ignore red via `stopsAtRedLight() -> !sirenOn`. Other traffic yields via a separate sensor check. Simpler design - the light state machine stays deterministic.

Why is `Intersection` sealed? Two concrete kinds exist: `SignalisedIntersection` and `Roundabout` (currently unused). Sealing prevents a stray third kind sneaking in and breaking the sensor's assumption that it only needs to handle these two.

Constants that matter to you

Constant	Value	Meaning
<code>STOP_LINE_RADIUS</code>	52	Where the sensor treats "before the light" - matters for your yellow-vs-red decision timing.
<code>ADAPTIVE_QUEUE_HIGH</code>	4	Vehicles queued before green may extend.
(grid.txt) <code>greenTicks</code> / <code>yellowTicks</code> / <code>redTicks</code>	per intersection	Base cycle durations, set in the map file.

Patterns you're defending

- **State** - `LightState` + `GreenState` / `YellowState` / `RedState`
- **Sealed hierarchy** - `Intersection` closed at compile time
- **Encapsulation** - `TrafficLight` hides `timer` and `state`; only `updateSignals` and `phaseFor` are exposed

What to skip in the main PDF

- Section 4.1 ROAD / BUS_STOP lines - Pair B; you only care about INTERSECTION lines
- Section 5 (Lane geometry) - Pair B's
- Section 7 (Vehicle movement) - Pair C's
- Section 8.2-8.3 (same-lane spacing / dot product) - Pair A's sensor
- Section 10 (Spawner + WeightedRandom) - Pair A's
- Section 11 (rendering) - Pair A's view